

Automated Research Systems

A Survey of Architectures, Approaches, and Trade-Offs
in LLM-Driven Research Automation

M. Berends

Claude (Anthropic)

March 2026

Abstract

Large language models have enabled a new class of systems that automate aspects of the research process—from literature discovery and synthesis to hypothesis generation, experimental optimisation, and full paper writing. This survey examines the major approaches to automated research (“autoresearch”), organising them into four categories: *deep research* (multi-step web search and report generation), *literature agents* (scientific question answering over academic corpora), *full-pipeline science* (end-to-end idea-to-paper systems), and *empirical optimisation* (autonomous code-level edit–benchmark–iterate loops). For each category we analyse the architectures and design decisions of leading systems, identify shared patterns and key differentiators, and discuss the trade-offs that shape system behaviour. We find that decomposition strategy, quality-control mechanisms, and context management are the principal axes along which systems diverge, and that open-source implementations have reached or exceeded commercial systems on several benchmarks.

Introduction

The application of LLMs to research automation has expanded rapidly since 2023. Early systems used retrieval-augmented generation (RAG) for single-turn question answering; current systems employ multi-agent architectures, iterative search loops, and tool use to conduct extended autonomous research sessions lasting minutes to hours.

Despite this proliferation, the landscape lacks a unifying framework. Systems marketed under similar labels—“deep research”, “AI scientist”, “research agent”—often automate fundamentally different aspects of the research process. A literature synthesis agent that finds and cites papers has little architectural overlap with an evolutionary code optimiser that discovers new algorithms through mutation and fitness evaluation.

This survey proposes a four-category taxonomy based on *what is being automated* (Section 2), then provides detailed analysis of exemplar systems in each category (Sections 3–6). Section 7 identifies cross-cutting patterns, and Section 8 discusses implications for future system design.

Taxonomy

We organise automated research systems along two dimensions: the *input* (what triggers the system) and the *output* (what it produces). This yields four natural categories: These categories are not mutually exclusive. The most ambitious systems (e.g. FutureHouse Robin) aim to span multiple categories by combining literature synthesis with hypothesis generation and experimental validation.

Category	Input	Output	Exemplars
Deep Research	Natural-language question	Multi-page cited report	OpenAI, Gemini, Perplexity, STORM, GPT-Researcher
Literature Agents	Research question	Cited answer with evidence	PaperQA2, OpenScholar, Elicit
Full-Pipeline Science	Domain + seed ideas	Paper, code, results	AI Scientist, ShinkaEvolve, Coscientist, SciAgents
Empirical Optimisation	Objective + codebase	Optimised code + log	pi-autoresearch, ShinkaEvolve, AlphaEvolve

Table 1: Taxonomy of automated research systems. ShinkaEvolve appears in two categories as it combines evolutionary algorithm discovery (full-pipeline) with code-level optimisation (empirical).

Deep Research Systems

Deep research systems accept a natural-language question and produce a comprehensive, cited report after conducting autonomous multi-step web search. All share a common architectural pattern—the iterative *ReAct loop* (plan → search → read → reason → repeat → synthesise)—but diverge significantly in depth, speed, and search strategy.

Commercial Systems

OpenAI Deep Research.

Launched February 2025 as a ChatGPT feature, OpenAI Deep Research is powered by a fine-tuned o3-class reasoning model trained via reinforcement learning on simulated browsing tasks. It executes 30–60 web searches per query across 150–200 reasoning iterations, visiting dozens of pages over 5–30 minutes. A secondary o3-mini model summarises the chain-of-thought for user display. At launch it achieved 51.5% on BrowseComp [OpenAI, 2025]—versus 1.9% for GPT-4o with browsing—and 26.6% on Humanity’s Last Exam. Its primary limitation is latency: the system optimises for depth at the expense of speed, with queries routinely taking 20+ minutes.

Google Gemini Deep Research.

Launched December 2024 on Gemini 2.0 Flash Thinking (later Gemini 3 Pro), Google’s system introduces an *asynchronous task manager* that maintains shared state between a planner model and parallel task-execution models. This enables graceful recovery from individual sub-task failures without restarting the entire research session. The user can review and edit the research plan before execution. Native Google Search integration provides arguably the highest-quality search backend of any system. On DRACO [Perplexity AI, 2026], Gemini Deep Research scores 58.97%, and on BrowseComp, Gemini-3.1-Pro reaches 85.9.

Perplexity Deep Research.

Perplexity optimises for the opposite end of the speed–depth trade-off. Using a modular planner–retriever–synthesiser architecture with 3–5 sequential refinement cycles, it completes most queries in under 3 minutes—approximately $9\times$ faster than OpenAI. It references ~ 50 sources per query on average (vs. ~ 20 for competitors) via on-demand web crawling rather than static indexes. As of February 2026, it integrates Claude Opus as the underlying reasoning model and achieves the

highest DRACO score of any system at 70.5% [Perplexity AI, 2026]. Perplexity also published the DRACO benchmark itself, designed to evaluate cross-domain deep research quality.

Grok DeepSearch / DeeperSearch.

xAI’s Grok 4.20 Beta introduced a native 4-agent multi-agent architecture: a coordinator (Grok), a research specialist (Harper), a logic/code specialist (Benjamin), and a creative/divergent specialist (Lucas). All four share the same $\sim 3T$ parameter MoE backbone with lightweight LoRA persona adapters. Multi-Agent Reinforcement Learning (MARL) during post-training rewards convergence on truthful answers. Uniquely, Grok integrates real-time X/Twitter data, giving it access to breaking news and social sentiment unavailable to other systems.

Comparison.

Table 2 summarises the key parameters.

System	Searches	Time	BrowseComp	DRACO
OpenAI Deep Research	30–60	5–30 min	51.5	52.1
Gemini Deep Research	100+ pages	3–10 min	85.9	59.0
Perplexity	~ 50 sources	1–3 min	—	70.5
Grok DeepSearch	10 iterations	2–3 min	—	—
Claude Research	multi-pass	5–45 min	84.0	—

Table 2: Commercial deep research systems. BrowseComp scores use latest available model variants. Dashes indicate unreported scores.

Open-Source Systems

GPT-Researcher.

GPT-Researcher [Elovic, 2024] uses a plan-and-execute pattern with a planner agent, researcher agent, and writer agent. The planner generates sub-queries from the research question; these are processed *concurrently* via `asyncio.gather`. Retrieval supports 12+ backends (Tavily, Google, Bing, DuckDuckGo, Serper, Arxiv, PubMed, Semantic Scholar, etc.). Scraped content is compressed via cosine-similarity filtering (threshold 0.35) over 1000-character chunks.

The system’s *deep research mode* implements recursive breadth-first exploration: at each level, it generates **breadth** queries (default 4), spawns nested **GPTResearcher** instances for each, extracts learnings and follow-up questions via LLM, and recurses to **depth** levels (default 2). Context is capped at 25,000 words. A separate multi-agent mode built on LangGraph adds role-specialised agents (editor, researcher, writer, reviewer, publisher) with optional human-in-the-loop review.

STORM.

STORM [Lu et al., 2024] (Stanford) takes a fundamentally different approach to question decomposition. Rather than generating sub-queries directly, it *simulates multi-expert dialogue*:

1. **Persona generation.** The system finds related Wikipedia topics, scrapes their tables of contents, and asks the LLM to create diverse “editor” personas (e.g. “Historian focusing on early development”, “Policy analyst examining regulation”).

2. **Conversation simulation.** For each persona, a multi-turn dialogue (default 3 turns) runs between a `WikiWriter` (the persona asking questions via `DSPy ChainOfThought`) and a `TopicExpert` (answering via retrieval). Conversations run concurrently.
3. **Outline and article generation.** The information table from all dialogues feeds hierarchical outline generation (merging a “direct” and “informed” outline), then section-by-section article writing with inline citations.
4. **Polishing.** A dedicated refinement pass handles deduplication and summarisation.

STORM uses five separate LLM configurations (conversation, question asking, outline generation, article writing, polishing), reflecting a design philosophy where different cognitive tasks benefit from different model sizes and prompting strategies. Co-STORM [Jiang et al., 2024] extends this with collaborative human-in-the-loop discourse.

The multi-perspective approach is the most architecturally novel decomposition strategy in this category: by simulating *who would care about this topic*, it produces more diverse coverage than direct sub-query generation.

MiroThinker.

MiroThinker [MiroMind AI, 2025] (MiroMind AI) achieves the highest reported BrowseComp score of any system: MiroThinker-H1 reaches 88.2, surpassing Gemini-3.1-Pro (85.9) and Claude-4.6-Opus (84.0). Limited architectural details are publicly available; the system is open source on GitHub.

Literature Agents

Literature agents specialise in answering questions grounded in academic publications. Unlike deep research systems that search the open web, these systems operate over curated corpora of scientific papers, with strong emphasis on citation faithfulness and structured evidence extraction.

PaperQA2

PaperQA2 [Lala et al., 2024] (FutureHouse) models research as a reinforcement learning problem. The `PaperQAEEnvironment` class implements an RL-style `Environment` with `reset()` and `step(action)` methods, returning observations, rewards, and termination signals.

The agent selects from five tools at each timestep:

PaperSearch	Queries a pre-built search index, adds papers to the working Docs collection.
GatherEvidence	Retrieves relevant chunks from loaded papers, scores each chunk 0–10 for relevance via a summary LLM, and stores scored <code>Context</code> objects.
GenerateAnswer	Synthesises an answer from top-ranked contexts with inline citations.
Reset	Clears all gathered evidence (recovery from bad search directions).
Complete	Terminates the session with a certainty assessment.

Three model roles serve different functions: a main LLM for answer generation, a summary LLM for evidence scoring, and an embedding model for semantic search. Paper metadata is enriched via four clients (Semantic Scholar, CrossRef, OpenAlex, Unpaywall), and a retraction checker flags withdrawn papers.

On LitQA2, PaperQA2 achieves **superhuman accuracy** (~60% vs. ~50% for PhD-level human experts) with very low hallucination rates—the strongest evidence that tool-augmented LLM agents can exceed human performance on academic question answering when properly grounded in source material.

OpenScholar

OpenScholar [Asai et al., 2024] (AI2) takes a different approach: rather than dynamic tool use, it performs dense retrieval over 45M+ full-text paper passages, reranks results, and synthesises answers with a fine-tuned 8B model specifically trained for citation faithfulness. On ScholarBench, this 8B model matches or exceeds GPT-4o and PaperQA2 on four scientific QA datasets while using a fraction of the compute. The key insight is that domain specialisation and retrieval augmentation can substitute for model scale in well-defined scientific QA tasks.

Elicit

Elicit (Ought/Elicit Inc.) focuses on *structured extraction* rather than free-form QA. It searches 200M+ papers via Semantic Scholar, then uses LLMs to extract structured data columns (study design, sample size, methods, findings) from abstracts and full texts. This makes it uniquely suited for systematic reviews and meta-analyses, where the goal is not a narrative answer but a structured comparison across studies. It is the most workflow-oriented system in this category, functioning as an assisted tool rather than an autonomous agent.

Consensus

Consensus indexes 200M+ academic papers and answers questions by extracting and aggregating claims from abstracts. Its signature feature is a “consensus meter” showing the degree of scientific agreement on a given claim across studies. A GPT-4-powered synthesis module (“Consensus Copilot”) provides narrative summaries. The system demonstrates that useful research automation need not require deep autonomy—sometimes aggregation across many simple extractions is more valuable than sophisticated reasoning over a few sources.

Full-Pipeline Science

Full-pipeline systems attempt to automate the complete research cycle: from idea generation through experimentation to paper writing and review. These are the most ambitious—and most controversial—automated research systems.

The AI Scientist

The AI Scientist [Lu et al., 2024] (Sakana AI) implements a five-phase sequential pipeline:

1. **Idea generation.** The LLM generates research ideas as structured JSON (name, title, experiment plan, novelty/feasibility ratings), with up to 5 rounds of self-reflection. All previously generated ideas are included in the prompt to encourage diversity.
2. **Novelty check.** A multi-round interactive loop (up to 10 iterations) where the LLM searches Semantic Scholar, reviews results, and decides whether to search again or declare the idea novel/not novel.
3. **Experimentation.** Uses Aider (an AI coding assistant) to modify `experiment.py`, run it, and iterate (up to 5 runs, 4 retries each). The system edits real code and runs real experiments.
4. **Writeup.** Section-by-section L^AT_EX generation with per-section prompts, a citation loop (up to 20 rounds of Semantic Scholar search and BibTeX integration), and two full refinement passes. Compilation errors are auto-corrected via Aider.
5. **Review.** Simulated NeurIPS-style peer review with ensemble of 5 reviews (temperature 0.1), scoring originality, quality, clarity, significance, and soundness.

The system produces full L^AT_EX papers compiled to PDF for ~\$15 per paper (with Claude Sonnet 3.5). Papers score ~4–5/10 on the automated reviewer—borderline reject at best venues. The primary limitation is quality: while technically correct, the papers lack novelty and depth.

The system is also restricted to ML/DL domains where experiments are code-runnable; it cannot perform wet-lab science.

Nevertheless, the AI Scientist demonstrates that end-to-end research automation is technically feasible, and its open-source release has enabled significant follow-on work.

ShinkaEvolve

ShinkaEvolve [Sakana AI, 2025] (Sakana AI) shifts from paper writing to *algorithm discovery* via evolutionary optimisation. The core loop maintains an archive of evaluated programs, uses LLMs to generate mutations and crossovers, and evaluates fitness against concrete metrics. Three innovations drive its sample efficiency:

Novelty-based rejection

Before expensive evaluation, program text embedding similarity plus an LLM-as-novelty-judge filters near-duplicates and trivial variations.

Multi-armed bandit LLM selection

A bandit dynamically picks the best model from an ensemble as evolution progresses, adapting to problem structure.

Exploration/exploitation balancing

An intelligent parent sampling strategy decides when to refine known solutions versus try novel directions.

Results are striking: ShinkaEvolve found a state-of-the-art circle packing solution ($n = 26$) in only 150 samples, where prior systems (including Google DeepMind’s AlphaEvolve) required thousands. It evolved a 3-stage agentic architecture for AIME math reasoning in 75 generations, and outperformed DeepSeek’s Global Load Balancing Loss for Mixture-of-Experts routing in 30 generations. The system is fully open source under Apache 2.0, in contrast to AlphaEvolve.

ShinkaEvolve’s key insight is that LLM-guided search over program space is dramatically more sample-efficient than random mutation when combined with novelty filtering—the LLM’s prior knowledge about what constitutes a “meaningfully different” program acts as an implicit regulariser.

Coscientist and ChemCrow

Coscientist [Boiko et al., 2023] (CMU, published in *Nature*) and ChemCrow [Bran et al., 2024] (EPFL) extend automated research into wet-lab chemistry. Both use tool-augmented LLM agents: ChemCrow integrates 18 chemistry tools (RDKit, safety databases, reaction predictors) in a ReAct-style loop; Coscientist additionally controls robotic lab equipment (Opentrons) to autonomously plan and execute chemical experiments. Coscientist successfully optimised palladium-catalysed cross-coupling reactions without human intervention.

These systems demonstrate that the agent paradigm extends beyond information tasks to physical-world experimentation, though they require specialised infrastructure and raise safety concerns around autonomous chemical synthesis.

SciAgents

SciAgents [Buehler, 2024] (MIT) takes a unique approach to hypothesis generation: rather than searching text, it reasons over *knowledge graphs*. Multiple LLM agents with different roles (ontologist, scientists, critic) interact over a graph of domain concepts. The ontology agent structures knowledge; scientist agents propose hypotheses by finding novel connections in the graph; a critic agent evaluates plausibility. This graph-based approach is particularly suited to materials science, where novel material properties often arise from unexpected combinations of known components.

Empirical Optimisation

Empirical optimisation systems automate the *edit–benchmark–iterate* cycle: they modify code, measure the effect, keep improvements, discard regressions, and repeat. Unlike the previous categories, these systems generate *new knowledge through experimentation* rather than synthesising existing knowledge.

pi-autoresearch

pi-autoresearch [Zechner and Beorlegui, 2025] is an extension for the `pi` AI coding agent that adds an autonomous experiment loop. Inspired by Karpathy’s autoresearch concept, it targets optimisation tasks with clear metrics: test speed, bundle size, ML training loss, build times, Lighthouse scores.

Architecture.

The system has three layers:

Extension	Registers three tools into the agent’s namespace: <code>init_experiment</code> (once), <code>run_experiment</code> (benchmark), and <code>log_experiment</code> (keep/discard). Manages session state and TUI dashboard.
Skills	Prompt-based instructions (<code>SKILL.md</code> files) that teach the agent the full workflow, including how to create benchmark scripts, correctness checks, and the loop rules.
Finalize	A bash script that splits a messy experiment branch into clean, independently-mergeable PR branches, verifying that their union matches the original.

Context management.

pi-autoresearch addresses the context exhaustion problem—the central challenge for long-running agent sessions—through several mechanisms:

- **Append-only JSONL log.** Every experiment result (metric values, status, description, confidence) is persisted regardless of whether the code change was kept or discarded.
- **ASI (Actionable Side Information).** Structured key–value annotations on each experiment that survive code reverts. When the agent’s context is reset, ASI prevents it from repeating failed experiments—the code is gone but the learnings persist.
- **Token-aware auto-resume.** The system tracks token consumption per iteration, predicts when the context window will overflow (with $1.2\times$ safety margin), gracefully stops, and auto-resumes in a fresh context up to 20 times. This enables multi-hour unattended optimisation sessions.
- **Output truncation.** Benchmark output sent to the LLM is capped at 10 lines / 4 KB, with structured `METRIC name=value` lines parsed automatically.

Quality control.

Confidence scoring uses Median Absolute Deviation (MAD): the ratio $|best_delta|/MAD$ across all runs distinguishes real improvements ($\geq 2.0\times$) from benchmark noise ($< 1.0\times$). Optional *backpressure checks* (`autoresearch.checks.sh`) run correctness validation (tests, types, lint) after each passing benchmark, preventing optimisation that breaks correctness.

Relationship to ShinkaEvolve and AlphaEvolve

ShinkaEvolve (Section 5) operates in the same edit–evaluate–iterate paradigm but at a different level of abstraction. Where pi-autoresearch optimises *within a codebase* (making incremental

changes to existing code), ShinkaEvolve evolves *entire algorithms* (generating novel programs from scratch or via crossover). AlphaEvolve (Google DeepMind, closed-source) occupies a similar niche to ShinkaEvolve but requires orders of magnitude more evaluations.

The key architectural difference is *what persists between iterations*: pi-autoresearch persists the codebase itself (via git commits) plus structured learnings (ASI); ShinkaEvolve persists a population archive with fitness scores; AlphaEvolve persists an evolutionary island model. All three demonstrate that LLMs are effective mutation operators for code, but the surrounding infrastructure—novelty filtering, confidence scoring, context management—determines practical utility.

Cross-Cutting Analysis

Decomposition Strategy

How a system breaks a research question into sub-tasks is the single most consequential architectural decision. We observe five distinct strategies:

Strategy	Systems	Trade-off
LLM sub-query generation	GPT-Researcher, OpenAI, Perplexity	Simple and fast; queries cluster around obvious angles
Multi-perspective simulation	STORM	Broader coverage; slower; more creative
Interactive novelty search	AI Scientist	Good for finding literature gaps; requires domain API
Evolutionary mutation	ShinkaEvolve	Efficient solution-space exploration; needs fitness function
Dynamic agent decision	PaperQA2	Most flexible; hardest to debug and control

Table 3: Decomposition strategies across systems.

The Speed–Depth Trade-Off

The central design tension in all automated research systems is how much compute to spend per query. This manifests differently across categories:

- **Deep research:** Perplexity/Grok (1–3 min, broad coverage) vs. OpenAI (5–30 min, deep reasoning).
- **Literature agents:** OpenScholar (single-pass retrieval + fine-tuned model) vs. PaperQA2 (multi-step tool use).
- **Full-pipeline:** AI Scientist (~\$15/paper, moderate quality) vs. human researchers (~months/paper, high quality).
- **Empirical optimisation:** pi-autoresearch (hours of autonomous iteration) vs. ShinkaEvolve (150 samples to SOTA).

No system has resolved this trade-off; the optimal point depends entirely on the use case. Speed-optimised systems excel at factual lookups and current-events research; depth-optimised systems excel at complex analytical questions requiring multi-hop reasoning.

Quality Control Mechanisms

The mechanisms systems use to ensure output quality are the strongest predictors of practical utility:

Mechanism	Systems	Effect
Relevance scoring per chunk	PaperQA2	Prevents low-quality evidence from diluting answers
Multi-perspective diversity	STORM	Prevents tunnel vision and query clustering
Novelty rejection sampling	ShinkaEvolve	Prevents wasted compute on near-duplicates
Backpressure checks	pi-autoresearch	Prevents correctness regressions during optimisation
Confidence via MAD	pi-autoresearch	Distinguishes signal from benchmark noise
AI peer review	AI Scientist	Catches errors post-hoc (weak calibration)
Retraction checking	PaperQA2	Flags withdrawn papers before citation

Table 4: Quality control mechanisms and their effects.

Context Management

Long-running agent sessions inevitably exhaust context windows. Systems address this in three ways:

1. **Large windows.** Commercial systems use 128K–1M token contexts (Gemini, Claude). Effective but expensive, and still insufficient for multi-hour sessions.
2. **Compression.** GPT-Researcher caps deep research at 25K words; PaperQA2 uses relevance scoring to prioritise what enters context.
3. **External memory.** pi-autoresearch uses append-only logs, ASI annotations, and session documents to persist learnings across context resets. This is the most sophisticated approach and the only one that enables truly multi-hour autonomous sessions.

Multi-Model Architectures

Most systems use multiple LLM configurations for different cognitive tasks. STORM uses five (conversation, question asking, outline, article writing, polishing). GPT-Researcher uses two (strategic for planning, smart for writing). PaperQA2 uses three (main, summary, embedding). ShinkaEvolve dynamically selects from an ensemble via multi-armed bandit.

The pattern suggests that *no single model is optimal for all sub-tasks* in a research workflow: lightweight models suffice for evidence scoring and query generation, while capable models are needed for synthesis and reasoning.

Open Source vs. Commercial

Open-source systems have reached or exceeded commercial systems on key benchmarks:

- **BrowseComp:** MiroThinker-H1 (88.2) > all commercial systems.
- **Algorithm discovery:** ShinkaEvolve (150 samples) beats AlphaEvolve (thousands of samples).
- **Scientific QA:** OpenScholar 8B matches GPT-4o on ScholarBench.
- **Literature QA:** PaperQA2 exceeds human PhD-level performance on LitQA2.

The commercial advantage lies in integration (search quality, UI, ecosystem) rather than raw capability. For researchers building custom systems, open-source components provide building blocks that match or exceed proprietary alternatives.

Conclusion

The automated research landscape has matured rapidly from single-turn RAG systems to multi-agent architectures capable of extended autonomous research sessions. Our four-category taxonomy—deep research, literature agents, full-pipeline science, and empirical optimisation—captures fundamentally different automation targets that share common architectural patterns but diverge in decomposition strategy, quality control, and context management.

Three findings stand out:

1. **Decomposition matters most.** STORM’s multi-perspective simulation produces more diverse research than direct sub-query generation; ShinkaEvolve’s novelty-filtered evolution is dramatically more sample-efficient than random mutation. How a system breaks problems apart determines coverage, efficiency, and output quality.
2. **Quality control is the differentiator.** The gap between a useful and useless research agent is not the underlying LLM but the surrounding quality infrastructure: relevance scoring (PaperQA2), confidence estimation (pi-autoresearch), novelty filtering (ShinkaEvolve), and perspective diversity (STORM).
3. **The two paradigms will converge.** Information synthesis (finding existing knowledge) and empirical discovery (generating new knowledge through experimentation) are currently separate. The most impactful future systems will combine both—using literature agents to generate hypotheses, then empirical optimisation to test them. FutureHouse Robin and the AI Scientist are early attempts, but neither does both halves well yet.

For practitioners building automated research systems, the evidence favours composing specialised open-source components (PaperQA2 for literature, STORM-style decomposition for breadth, pi-autoresearch-style context management for long sessions) over relying on monolithic commercial offerings. The tools are available; the remaining challenge is integration.

References

- Boiko, D. A., MacKnight, R., and Gomes, G. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.
- Bran, A. M., Cox, S., Schilter, O., Baldassari, C., White, A. D., and Schwaller, P. ChemCrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024.
- Elovic, A. GPT-Researcher: Autonomous agent for comprehensive online research. <https://github.com/assafelovic/gpt-researcher>, 2024.
- Lu, C., Lu, Y., Lange, R. T., Foerster, J., Clune, J., and Ha, D. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- Lu, Y., et al. STORM: Synthesis of topic outlines through retrieval and multi-perspective question asking. *arXiv preprint arXiv:2402.14207*, 2024.
- Jiang, Y., et al. Into the unknown unknowns: Engaged human learning through participation in language model agent conversations. *arXiv preprint arXiv:2408.15232*, 2024.
- Lala, J., O’Donoghue, O., Mondal, A., Wallace, B., and White, A. D. PaperQA: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2024.
- Asai, A., et al. OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs. *arXiv preprint arXiv:2411.14199*, 2024.
- Buehler, M. J., et al. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024.

- Sakana AI. ShinkaEvolve: Evolutionary code optimization with LLMs. *arXiv preprint arXiv:2509.19349*, 2025.
- Zechner, M. and Beorlegui, D. pi-autoresearch: Autonomous experiment loops for AI coding agents. <https://github.com/davebcn87/pi-autoresearch>, 2025.
- OpenAI. BrowseComp: A simple challenge for browsing agents. <https://openai.com/index/browsecomp/>, 2025.
- MiroMind AI. MiroThinker: Open research agent. <https://github.com/MiroMindAI/MiroThinker>, 2025.
- Perplexity AI. DRACO: Evaluating deep research performance in the wild. *arXiv preprint arXiv:2602.11685*, 2026.